

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

STAWAN HITESH RUIWALE (1WA24CS241)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by STAWAN HITESH RUIWALE (1WA24CS241) who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Associate Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Table of Contents

Sl. No.	Experiment Title	Pg. No.
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	1
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories - system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS	21
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	26
4	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	36
5	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	42

6	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	49
7	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU	53

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Lab Program 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

- a) FCFS
- b) SJF
- c) Priority
- d) Round Robin

a) FCFS

Program:

```
#include <stdio.h>
```

```
int main() {
    int n, curr_time = 0, pid[10], at[10], bt[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, temp;
    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;

        printf("Enter arrival time for process[%d]: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process[%d]: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // sorting the processes by AT
    for (i = 0; i < n; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                // swapping PID, AT, and BT
                temp = pid[j];
                pid[j] = pid[j + 1];
                pid[j + 1] = temp;
                temp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = temp;
            }
        }
    }

    // simulation logic for FCFS would go here
}
```

```

        at[j + 1] = temp;
        temp = bt[j];
        bt[j] = bt[j + 1];
        bt[j + 1] = temp;
    }
}
}

// CT
for (i = 0; i < n; i++) {
    // CPU idle time
    if (curr_time < at[i]) {
        curr_time = at[i];
    }

    // calculating start time for RT
    st[i] = curr_time;

    curr_time += bt[i];
    ct[i] = curr_time;
}

// TAT
for (i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    tat_sum += tat[i];
}

// WT
for (i = 0; i < n; i++) {
    wt[i] = tat[i] - bt[i];
    wt_sum += wt[i];
}

// RT
for (i = 0; i < n; i++) {
    rt[i] = st[i] - at[i];
}

// header row
printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

```

```

// sorted by PID
for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
        if (pid[i] == j)
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
                tat[i], wt[i], rt[i]);
    }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
printf("Average WT: %.2f\n", (wt_sum / n));

return 0;
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

```

```

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6

```

P	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P2	8	3	20	12	9	9
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6

```

Average TAT: 9.75
Average WT: 4.75

```

```

○ stavvv@Stavs-MacBook-Air OS_LAB_PGMS %

```

b) SJF

Program:

```
#include <stdio.h>
```

```
int main() {
    int n, curr_time = 0, pid[10], at[10], bt[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, temp, completed = 0, done[10] = {0};
    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;

        printf("Enter arrival time for process[%d]: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process[%d]: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // sorting the processes by AT
    for (i = 0; i < n; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                // swapping PID, AT, and BT
                temp = pid[j];
                pid[j] = pid[j + 1];
                pid[j + 1] = temp;
                temp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = temp;
                temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;
            }
        }
    }

    while (completed != n) {
```



```

int min = 99999999, // to set the lowest remaining time
    s = -1;        // index of the process with the shortest remaining time

for (i = 0; i < n; i++) {
    // 1. AT is now or before current time
    // 2. check if the process is not completed
    // 3. least burst time
    if (at[i] <= curr_time && !done[i] && bt[i] < min) {
        min = bt[i];
        s = i;
    }
}

// CPU idle time
if (s == -1) {
    curr_time++;
    continue;
}

// set the start time
st[s] = curr_time;

curr_time += bt[s];

// CT
ct[s] = curr_time;

// TAT
tat[s] = ct[s] - at[s];
tat_sum += tat[s];

// WT
wt[s] = tat[s] - bt[s];
wt_sum += wt[s];

// RT
rt[s] = st[s] - at[s];

// increment the completed processes count
completed++;
done[s] = 1;

```

```

}

// header row
printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
        if (pid[i] == j)
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
                tat[i], wt[i], rt[i]);
    }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
printf("Average WT: %.2f\n", (wt_sum / n));

return 0;
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6

P      AT      BT      CT      TAT      WT      RT
P1      0       7       7       7       0       0
P2      8       3      14       6       3       3
P3      3       4      11       8       4       4
P4      5       6      20      15       9       9

Average TAT: 9.00
Average WT: 4.00
stavvv@Stavs-MacBook-Air OS_LAB_PGMS %

```

c) SRTF

Program:

```
#include <stdio.h>
```

```
int main() {
    int n, completed = 0, curr_time = 0, pid[10], at[10], bt[10], remt[10],
        ct[10], tat[10], wt[10], st[10], rt[10], i, j;
    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;

        printf("Enter arrival time for process[%d]: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process[%d]: ", i + 1);
        scanf("%d", &bt[i]);
        remt[i] = bt[i]; // initialising remaining time
    }

    // running a while loop for each time interval
    // until all the processes have completed
    while (completed != n) {

        int min = 99999999, // to set the lowest remaining time
            s = -1;         // index of the process with the shortest remaining time

        for (i = 0; i < n; i++) {
            // 1. AT is now or before current time
            // 2. process has some remaining time
            // 3. least remaining time
            if (at[i] <= curr_time && remt[i] > 0 && remt[i] < min) {
                min = remt[i];
                s = i;
            }
        }

        // CPU idle time
```

```

if (s == -1) {
    curr_time++;
    continue;
}

// if the process is in the CPU for the first time
if (remt[s] == bt[s])
    st[s] = curr_time;

// decrement the remaining time of the process
remt[s]--;

// if the process has completed
if (remt[s] == 0) {
    // CT
    ct[s] = curr_time + 1;

    // TAT
    tat[s] = ct[s] - at[s];
    tat_sum += tat[s];

    // WT
    wt[s] = tat[s] - bt[s];
    wt_sum += wt[s];

    // RT
    rt[s] = st[s] - at[s];

    // increment the completed processes count
    completed++;
}
curr_time++;
}

// header row
printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
        if (pid[i] == j)

```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
            tat[i], wt[i], rt[i]);
    }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
printf("Average WT: %.2f\n", (wt_sum / n));
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN: 1WA24CS241

```

```

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6

```

P	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P2	8	3	11	3	0	0
P3	3	4	14	11	7	4
P4	5	6	20	15	9	9

```

Average TAT: 9.00
Average WT: 4.00

```

```

stavvv@Stavs-MacBook-Air OS_LAB_PGMS % █

```

d) Priority Non-Preemptive

Program:

```
#include <stdio.h>
```

```
int main() {
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

    int n, curr_time = 0, pid[10], at[10], bt[10], p[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, completed = 0, done[10] = {0}, choice;
    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;

        printf("Enter arrival time for process[%d]: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process[%d]: ", i + 1);
        scanf("%d", &bt[i]);

        printf("Enter priority for process[%d]: ", i + 1);
        scanf("%d", &p[i]);
    }

    printf("\nType:");
    printf("1. Lower the number higher the priority\n");
    printf("2. Higher the number higher the priority\n");
    scanf("%d", &choice);

    while (completed != n) {

        int min = 99999999, // to set with the lowest priority
            max = -99999999, // to set with the highest priority
            s = -1; // index of the process with the lowest/highest priority

        for (i = 0; i < n; i++) {
```

```

// 1. AT is now or before current time
// 2. priority is lesser than min
// 3. process is not completed
if (choice == 1) {
    if (at[i] <= curr_time && p[i] < min && !done[i]) {
        min = p[i];
        s = i;
    }
}
if (choice == 2) {
    if (at[i] <= curr_time && p[i] > max && !done[i]) {
        max = p[i];
        s = i;
    }
}
}
}

```

```

// CPU idle time

```

```

if (s == -1) {
    curr_time++;
    continue;
}

```

```

st[s] = curr_time;
curr_time += bt[s];

```

```

// CT
ct[s] = curr_time;

```

```

// TAT
tat[s] = ct[s] - at[s];
tat_sum += tat[s];

```

```

// WT
wt[s] = tat[s] - bt[s];
wt_sum += wt[s];

```

```

// RT
rt[s] = st[s] - at[s];

```

```

// increment the completed processes count

```

```

    completed++;
    done[s] = 1;
}

// header row
printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
        if (pid[i] == j)
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], p[i], ct[i],
                tat[i], wt[i], rt[i]);
    }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
printf("Average WT: %.2f\n", (wt_sum / n));

return 0;
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter priority for process[1]: 3
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter priority for process[2]: 1
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter priority for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6
Enter priority for process[4]: 2

Type:1. Lower the number higher the priority
2. Higher the number higher the priority
1

```

P	AT	BT	PR	CT	TAT	WT	RT
P1	0	7	3	7	7	0	0
P2	8	3	1	16	8	5	5
P3	3	4	4	20	17	13	13
P4	5	6	2	13	8	2	2

```

Average TAT: 10.00
Average WT: 5.00
stavvv@Stavs-MacBook-Air OS_LAB_PGMS %

```


e) Priority Preemptive

Program:

```
#include <stdio.h>
```

```
int main() {
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

    int n, curr_time = 0, pid[10], at[10], bt[10], remt[10], p[10], ct[10],
        tat[10], wt[10], st[10], rt[10], i, j, completed = 0, choice;
    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;

        printf("Enter arrival time for process[%d]: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process[%d]: ", i + 1);
        scanf("%d", &bt[i]);
        remt[i] = bt[i];

        printf("Enter priority for process[%d]: ", i + 1);
        scanf("%d", &p[i]);
    }

    printf("\nType:");
    printf("1. Lower the number higher the priority\n");
    printf("2. Higher the number higher the priority\n");
    scanf("%d", &choice);

    while (completed != n) {

        int min = 99999999, // to set with the lowest priority
            max = -99999999, // to set with the highest priority
            s = -1; // index of the process with the lowest/highest priority

        for (i = 0; i < n; i++) {
            // 1. AT is now or before current time
```

```

// 2. priority is lesser than min
// 3. there is some time remaining
if (choice == 1) {
    if (at[i] <= curr_time && p[i] < min && remt[i] > 0) {
        min = p[i];
        s = i;
    }
}
if (choice == 2) {
    if (at[i] <= curr_time && p[i] > max && remt[i] > 0) {
        max = p[i];
        s = i;
    }
}
}

```

```

// CPU idle time

```

```

if (s == -1) {
    curr_time++;
    continue;
}

```

```

// if the process is in the CPU for the first time

```

```

if (remt[s] == bt[s])
    st[s] = curr_time;

```

```

// decrement the remaining time of the process

```

```

remt[s]--;

```

```

// if the process has completed

```

```

if (remt[s] == 0) {
    // CT
    ct[s] = curr_time + 1;

```

```

    // TAT

```

```

    tat[s] = ct[s] - at[s];
    tat_sum += tat[s];

```

```

    // WT

```

```

    wt[s] = tat[s] - bt[s];
    wt_sum += wt[s];

```

```

// RT
rt[s] = st[s] - at[s];

// increment the completed processes count
completed++;
}
curr_time++;
}

// header row
printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
        if (pid[i] == j)
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], p[i], ct[i],
                tat[i], wt[i], rt[i]);
    }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
printf("Average WT: %.2f\n", (wt_sum / n));

return 0;
}

```

Output:

```
Name: STAWAN HITESH RUIWALE
USN:1WA24CS241
```

```
Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter priority for process[1]: 3
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter priority for process[2]: 1
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter priority for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6
Enter priority for process[4]: 2
```

```
Type:1. Lower the number higher the priority
2. Higher the number higher the priority
2
```

P	AT	BT	PR	CT	TAT	WT	RT
P1	0	7	3	11	11	4	0
P2	8	3	1	20	12	9	9
P3	3	4	4	7	4	0	0
P4	5	6	2	17	12	6	6

```
Average TAT: 9.75
```

```
Average WT: 4.75
```

```
stavvv@Stavs-MacBook-Air OS_LAB_PGMS %
```

f) Round Robin

Program:

```
#include <stdio.h>
```

```
int main() {
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

    int n, completed = 0, curr_time = 0, pid[10], at[10], bt[10], remt[10],
        ct[10], tat[10], wt[10], st[10], rt[10], i, j, tq, q[10], f = 0,
        r = -1, inq[10] = {0};
    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;

        printf("Enter arrival time for process[%d]: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process[%d]: ", i + 1);
        scanf("%d", &bt[i]);
        remt[i] = bt[i]; // initialising remaining time
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    // running a while loop for each time interval
    // until all the processes have completed
    while (completed != n) {

        // add processes that have arrived to queue
        for (i = 0; i < n; i++) {
            if (at[i] <= curr_time && remt[i] > 0 && !inq[i]) {
                r = (r + 1) % 10;
                q[r] = i;
                inq[i] = 1;
            }
        }
    }
}
```

```

// CPU idle time
if (f > r) {
    curr_time++;
    continue;
}

// dequeue
i = q[f];
f = (f + 1) % 10;
inq[i] = 0;

// if the process is in the CPU for the first time
if (remt[i] == bt[i])
    st[i] = curr_time;

// decrement the remaining time of the process
if (remt[i] < tq) {
    curr_time += remt[i];
    remt[i] = 0;
} else {
    curr_time += tq;
    remt[i] -= tq;
}

// add newly arrived processes during execution
for (j = 0; j < n; j++) {
    if (at[j] <= curr_time && remt[j] > 0 && !inq[j] && j != i) {
        r = (r + 1) % 10;
        q[r] = j;
        inq[j] = 1;
    }
}

// if the process has completed
if (remt[i] == 0) {
    // CT
    ct[i] = curr_time;

    // TAT
    tat[i] = ct[i] - at[i];
}

```

```

    tat_sum += tat[i];

    // WT
    wt[i] = tat[i] - bt[i];
    wt_sum += wt[i];

    // RT
    rt[i] = st[i] - at[i];

    // increment the completed processes count
    completed++;
} else {
    r = (r + 1) % 10;
    q[r] = i;
    inq[i] = 1;
}
}

// header row
printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
        if (pid[i] == j)
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
                tat[i], wt[i], rt[i]);
    }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
printf("Average WT: %.2f\n", (wt_sum / n));

return 0;
}

```

Output:

```
Name: STAWAN HITESH RUIWALE
USN:1WA24CS241
```

```
Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6
Enter time quantum: 3
```

P	AT	BT	CT	TAT	WT	RT
P1	0	7	17	17	10	0
P2	8	3	16	8	5	5
P3	3	4	13	10	6	0
P4	5	6	20	15	9	4

```
Average TAT: 12.50
```

```
Average WT: 7.50
```

```
stavvv@Stavs-MacBook-Air OS_LAB_PGMS %
```


Lab Program 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories - system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Program:

```
#include <stdio.h>
```

```
int main() {
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

    int n, curr_time = 0, i, j;
    int ns = 0, nu = 0;
    int type[10], tq;

    int completedSys = 0, completedUsr = 0;

    int q[10], f = 0, r = -1;
    int inq[10] = {0}, remt[10];

    int sysPID[10], sysAT[10], sysBT[10], sysCT[10], sysTAT[10], sysWT[10];
    int usrPID[10], usrAT[10], usrBT[10], usrCT[10], usrTAT[10], usrWT[10];

    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    // input
    for (i = 0; i < n; i++) {

        printf("Enter type for process[%d] (0=system, 1=user): ", i + 1);
```

```

scanf("%d", &type[i]);

printf("Enter AT and BT for process[%d]: ", i + 1);

if (type[i] == 0) {
    scanf("%d %d", &sysAT[ns], &sysBT[ns]);
    sysPID[ns] = i + 1;
    remt[ns] = sysBT[ns];
    ns++;
} else {
    scanf("%d %d", &usrAT[nu], &usrBT[nu]);
    usrPID[nu] = i + 1;
    nu++;
}
}

printf("Enter time quantum: ");
scanf("%d", &tq);

// run until both queues finished
while (completedSys < ns || completedUsr < nu) {

    // add system processes to RR queue
    for (i = 0; i < ns; i++) {
        if (sysAT[i] <= curr_time && remt[i] > 0 && !inq[i]) {
            r = (r + 1) % 10;
            q[r] = i;
            inq[i] = 1;
        }
    }

    // ----- check if system queue empty -----
    if (f == (r + 1) % 10) {

        int ranUser = 0;

        // run user FCFS if ready
        for (i = 0; i < nu; i++) {

            if (usrBT[i] > 0 && usrAT[i] <= curr_time) {

```

```

curr_time += usrBT[i];

usrCT[i] = curr_time;

usrTAT[i] = usrCT[i] - usrAT[i];
usrWT[i] = usrTAT[i] - usrBT[i];

tat_sum += usrTAT[i];
wt_sum += usrWT[i];

usrBT[i] = 0;
completedUsr++;

ranUser = 1;
break;
}
}

if (!ranUser)
    curr_time++;

continue;
}

// ----- run system RR -----
i = q[f];
f = (f + 1) % 10;
inq[i] = 0;

if (f == (r + 1) % 10) {
    f = 0;
    r = -1;
}

if (remt[i] < tq) {
    curr_time += remt[i];
    remt[i] = 0;
} else {
    curr_time += tq;
    remt[i] -= tq;
}

```

```

// add new arrivals
for (j = 0; j < ns; j++) {
    if (sysAT[j] <= curr_time && remt[j] > 0 && !inq[j]) {
        r = (r + 1) % 10;
        q[r] = j;
        inq[j] = 1;
    }
}

// finished
if (remt[i] == 0) {

    sysCT[i] = curr_time;

    sysTAT[i] = sysCT[i] - sysAT[i];
    sysWT[i] = sysTAT[i] - sysBT[i];

    tat_sum += sysTAT[i];
    wt_sum += sysWT[i];

    completedSys++;

} else {

    r = (r + 1) % 10;
    q[r] = i;
    inq[i] = 1;
}
}

// print
printf("\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n\n");

for (j = 1; j <= n; j++) {

    for (i = 0; i < ns; i++) {
        if (sysPID[i] == j) {
            printf("P%d\t%d\t%d\tSystem\t%d\t%d\t%d\n",
                sysPID[i], sysAT[i], sysBT[i],
                sysCT[i], sysTAT[i], sysWT[i]);
        }
    }
}

```

```

    }
}

for (i = 0; i < nu; i++) {
    if (usrPID[i] == j) {
        printf("P%d\t%d\t%d\tUser\t%d\t%d\t%d\n",
            usrPID[i], usrAT[i], usrBT[i],
            usrCT[i], usrTAT[i], usrWT[i]);
    }
}
}

printf("\nAverage TAT: %.2f\n", tat_sum / n);
printf("Average WT: %.2f\n", wt_sum / n);

return 0;
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN: 1WA24CS241

```

```

Enter no. of processes: 5
Enter type for process[1] (0=system, 1=user): 0
Enter AT and BT for process[1]: 0 5
Enter type for process[2] (0=system, 1=user): 1
Enter AT and BT for process[2]: 1 4
Enter type for process[3] (0=system, 1=user): 0
Enter AT and BT for process[3]: 2 6
Enter type for process[4] (0=system, 1=user): 1
Enter AT and BT for process[4]: 3 3
Enter type for process[5] (0=system, 1=user): 0
Enter AT and BT for process[5]: 5 4
Enter time quantum: 2

```

PID	AT	BT	Type	CT	TAT	WT
P1	0	5	System	7	7	2
P2	1	0	User	19	18	14
P3	2	6	System	13	11	5
P4	3	0	User	22	19	16
P5	5	4	System	15	10	6

```
Average TAT: 22.20
```

```
Average WT: 12.60
```

```
○ stavvv@Stavs-MacBook-Air OS_LAB_PGMS %
```

Lab Program 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

a) Rate Monotonic

Program:

```
#include <stdio.h>
```

```
struct Task {  
    int id;  
    int execution;  
    int period;  
    int remaining;  
    int next_release;  
    int priority;  
};
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}
```

```
int lcm(int a, int b) { return (a * b) / gcd(a, b); }
```

```
int main() {
```

```
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");
```

```
    int n, i, t, hyper = 1;
```

```
    printf("Enter number of tasks: ");  
    scanf("%d", &n);
```

```
    struct Task task[n];
```

```

for (i = 0; i < n; i++) {
    printf("\nTask %d Execution Time: ", i + 1);
    scanf("%d", &task[i].execution);

    printf("Task %d Period: ", i + 1);
    scanf("%d", &task[i].period);

    task[i].id = i + 1;
    task[i].remaining = 0;
    task[i].next_release = 0;
}

// Assign priority (smaller period = higher priority)
for (i = 0; i < n; i++) {
    task[i].priority = task[i].period;
}

// Calculate hyperperiod (LCM of all periods)
for (i = 0; i < n; i++) {
    hyper = lcm(hyper, task[i].period);
}

printf("\nHyperperiod = %d\n", hyper);

printf("\nTime\tRunning Task\n");

for (t = 0; t < hyper; t++) {
    // Release tasks
    for (i = 0; i < n; i++) {
        if (t == task[i].next_release) {
            task[i].remaining = task[i].execution;
            task[i].next_release += task[i].period;
        }
    }
}

int best = -1;

// Choose highest priority task (smallest period)
for (i = 0; i < n; i++) {
    if (task[i].remaining > 0) {

```

```

        if (best == -1 || task[i].priority < task[best].priority) {
            best = i;
        }
    }
}

if (best != -1) {
    printf("%d\tT%d\n", t, task[best].id);
    task[best].remaining--;
} else {
    printf("%d\tIdle\n", t);
}
}

return 0;
}

```


Output:

```
Name: STAWAN HITESH RUIWALE
USN:1WA24CS241
```

```
Enter number of tasks: 2
```

```
Task 1 Execution Time: 1
Task 1 Period: 4
```

```
Task 2 Execution Time: 2
Task 2 Period: 5
```

```
Hyperperiod = 20
```

Time	Running Task
0	T1
1	T2
2	T2
3	Idle
4	T1
5	T2
6	T2
7	Idle
8	T1
9	Idle
10	T2
11	T2
12	T1
13	Idle
14	Idle
15	T2
16	T1
17	T2
18	Idle
19	Idle

```
○ stavvv@Stavs-MacBook-Air OS_LAB_PGMS %
```

b) Earliest Deadline First

Program:

```
#include <stdio.h>
```

```
struct Task {  
    int id;  
    int execution;  
    int period;  
    int deadline; // relative deadline  
    int remaining;  
    int next_release;  
    int abs_deadline; // absolute deadline  
};
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}
```

```
int lcm(int a, int b) { return (a * b) / gcd(a, b); }
```

```
int main() {
```

```
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");
```

```
    int n, i, t, hyper = 1;
```

```
    printf("Enter number of tasks: ");  
    scanf("%d", &n);
```

```
    struct Task task[n];
```

```
    for (i = 0; i < n; i++) {  
        task[i].id = i + 1;
```

```
        printf("\nTask %d Execution Time: ", i + 1);
```

```

scanf("%d", &task[i].execution);

printf("Task %d Period: ", i + 1);
scanf("%d", &task[i].period);

printf("Task %d Deadline: ", i + 1);
scanf("%d", &task[i].deadline);

task[i].remaining = 0;
task[i].next_release = 0;
task[i].abs_deadline = 0;
}

// Hyperperiod
for (i = 0; i < n; i++) {
    hyper = lcm(hyper, task[i].period);
}

printf("\nHyperperiod = %d\n", hyper);
printf("\nTime\tRunning Task\n");

for (t = 0; t < hyper; t++) {
    // Release tasks
    for (i = 0; i < n; i++) {
        if (t == task[i].next_release) {
            task[i].remaining = task[i].execution;
            task[i].abs_deadline = t + task[i].deadline;
            task[i].next_release += task[i].period;
        }
    }
}

int best = -1;

// Choose earliest deadline
for (i = 0; i < n; i++) {
    if (task[i].remaining > 0) {
        if (best == -1 || task[i].abs_deadline < task[best].abs_deadline) {
            best = i;
        }
    }
}
}

```

```
if (best != -1) {  
    printf("%d\tT%d\n", t, task[best].id);  
    task[best].remaining--;  
} else {  
    printf("%d\tIdle\n", t);  
}  
}  
  
return 0;  
}
```

Output:

```
Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter number of tasks: 2

Task 1 Execution Time: 1
Task 1 Period: 4
Task 1 Deadline: 3

Task 2 Execution Time: 2
Task 2 Period: 5
Task 2 Deadline: 5

Hyperperiod = 20

Time    Running Task
0       T1
1       T2
2       T2
3       Idle
4       T1
5       T2
6       T2
7       Idle
8       T1
9       Idle
10      T2
11      T2
12      T1
13      Idle
14      Idle
15      T2
16      T1
17      T2
18      Idle
19      Idle

○ stavvv@Stavs-MacBook-Air OS_LAB_PGMS %
```

c) Proportional Scheduling

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
typedef struct {
    char name[5];
    int tickets;
} Process;
```

```
int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;
```

```
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));
```

```
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }
```

```
    printf("\n--- Proportional Share Scheduling ---\n");
    printf("Enter the Time Period for scheduling: ");
    int m;
    scanf("%d", &m);
```

```
    for (int i = 0; i < m; i++) {
        int winning_ticket = rand() % total_tickets + 1;
        int accumulated_tickets = 0;
        int winner_index;
```

```
        for (int j = 0; j < n; j++) {
            accumulated_tickets += p[j].tickets;
```

```

        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
        p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %.2f%% of Processor Time.\n", p[i].name,
        ((p[i].tickets / total_T) * 100));
}

return 0;
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 36, Winner: P3
Tickets picked: 52, Winner: P3
Tickets picked: 40, Winner: P3
Tickets picked: 29, Winner: P2
Tickets picked: 22, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
stavvv@Stavs-MacBook-Air OS_LAB_PGMS %

```

Lab Program 4

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

a) Producer-Consumer (Semaphores)

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;          // counts empty slots
sem_t full;            // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

// Producer function
void* producer(void* arg) {
    int item, i = 0;

    while (1) {
        item = i++; // produce an item

        sem_wait(&empty);          // wait for empty slot
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full);              // signal item available
    }
}
```



```

        sleep(1); // simulate production time
    }
}

// Consumer function
void* consumer(void* arg) {
    int item;

    while (1) {
        sem_wait(&full);           // wait for filled slot
        pthread_mutex_lock(&mutex); // enter critical section

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&empty);           // signal empty slot

        sleep(2); // simulate consumption time
    }
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
}

```

```
return 0; }
```

Output:

```
Name: STAWAN HITESH RUIWALE  
USN:1WA24CS241
```

```
Produced: 0 at buffer[0]  
Consumed: 0 from buffer[0]  
Produced: 1 at buffer[1]  
Consumed: 1 from buffer[1]  
Produced: 2 at buffer[2]  
Produced: 3 at buffer[3]  
Consumed: 2 from buffer[2]  
Produced: 4 at buffer[4]  
Produced: 5 at buffer[0]  
Consumed: 3 from buffer[3]  
Produced: 6 at buffer[1]  
Produced: 7 at buffer[2]  
Consumed: 4 from buffer[4]  
Produced: 8 at buffer[3]  
Produced: 9 at buffer[4]  
Consumed: 5 from buffer[0]  
Produced: 10 at buffer[0]  
Produced: 11 at buffer[1]  
Consumed: 11 from buffer[1]  
Produced: 12 at buffer[2]  
Produced: 13 at buffer[3]  
Consumed: 12 from buffer[2]  
Produced: 14 at buffer[4]  
Produced: 15 at buffer[0]  
Consumed: 13 from buffer[3]  
Produced: 16 at buffer[1]  
Produced: 17 at buffer[2]  
Consumed: 14 from buffer[4]  
Produced: 18 at buffer[3]  
Produced: 19 at buffer[4]
```

b) Dining-Philosopher (Semaphores)

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;          // left fork index
    int right = (id + 1) % N; // right fork index

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking

        // To avoid deadlock, philosophers with even id pick left then right,
        // odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else
        {
            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
    }
}
```

```

        // Pick up left fork
        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id, left);
    }

    // Eating
    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    // Put down forks
    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}

return NULL;
}

int main()
{
    int i;
    int ids[N];

    // Initialize forks (mutexes)
    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads (will never happen here, but good practice)
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }
}

```

```

}

// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
    pthread_mutex_destroy(&forks[i]);
}

return 0;
}

```

Output;

```

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

```

```

Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.

```

Lab Program 5

Write a C program to simulate:

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

a) Deadlock Avoidance

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");
```

```
    int n, m;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], avail[m];
```

```
    int need[n][m], finish[n], safeSeq[n];
```

```
    // Input Allocation Matrix
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    // Input Maximum Demand Matrix
```

```
    printf("Enter Maximum Demand Matrix:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &max[i][j]);
```

```
    // Input Available Resources
```

```
    printf("Enter Available Resources:\n");
```

```
    for (int j = 0; j < m; j++)
```

```
        scanf("%d", &avail[j]);
```

```

// Step 1: Calculate Need and initialize Finish
for (int i = 0; i < n; i++) {
    finish[i] = 0; // false
    for (int j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
}

// Work = Available
int work[m];
for (int j = 0; j < m; j++)
    work[j] = avail[j];

// Step 2 & 3: Safety Algorithm
int count = 0;
while (count < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (finish[i] == 0) {
            // Check if Need[i] <= Work
            int canRun = 1;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    canRun = 0;
                    break;
                }
            }
        }
    }
    // Step 3: Allocate resources and mark finished
    if (canRun) {
        for (int j = 0; j < m; j++)
            work[j] += alloc[i][j];
        finish[i] = 1; // true
        safeSeq[count++] = i;
        found = 1;
    }
}
if (!found)
    break; // No process found, not safe
}

```

```

// Step 4: Check if all finished
if (count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for (int i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
        if (i != n - 1)
            printf(" -> ");
    }
    printf("\n");
} else {
    printf("\nSystem is NOT in a safe state.\n");
}

return 0;
}

```

Output:

```

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
stavvv@Stavs-MacBook-Air OS_LAB_PGMS % 

```


b) Deadlock Detection

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");
```

```
    int n, m;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], request[n][m], avail[m];
```

```
    int finish[n], safeSeq[n];
```

```
    // Input Allocation Matrix
```

```
    printf("Enter the allocation matrix:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Process %d: ", i);
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    }
```

```
    // Input Request Matrix
```

```
    printf("Enter the request matrix:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Process %d: ", i);
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &request[i][j]);
```

```
    }
```

```
    // Input Available Resources
```

```
    printf("Enter the available resources: ");
```

```
    for (int j = 0; j < m; j++)
```

```
        scanf("%d", &avail[j]);
```

```
    // Step 1(a): Work = Available
```

```
    int work[m];
```

```

for (int j = 0; j < m; j++)
    work[j] = avail[j];

// Step 1(b): If Allocation[i] != 0, Finish[i] = false, else true
for (int i = 0; i < n; i++) {
    int hasAlloc = 0;
    for (int j = 0; j < m; j++) {
        if (alloc[i][j] != 0) {
            hasAlloc = 1;
            break;
        }
    }
    finish[i] = hasAlloc ? 0 : 1; // 0 = false, 1 = true
}

// Step 2 & 3: Detection Algorithm
int count = 0;
while (count < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (finish[i] == 0) {
            // Check if Request[i] <= Work
            int canRun = 1;
            for (int j = 0; j < m; j++) {
                if (request[i][j] > work[j]) {
                    canRun = 0;
                    break;
                }
            }
            // Step 3: Release resources, mark finished
            if (canRun) {
                for (int j = 0; j < m; j++)
                    work[j] += alloc[i][j];
                finish[i] = 1;
                safeSeq[count++] = i;
                found = 1;
            }
        }
    }
    if (!found)
        break;
}

```

```

}

// Step 4: Check for deadlock
int deadlock = 0;
for (int i = 0; i < n; i++) {
    if (finish[i] == 0) {
        deadlock = 1;
        break;
    }
}

if (!deadlock) {
    printf("System is in safe state.\n");
    printf("Safe Sequence is: ");
    for (int i = 0; i < count; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");
} else {
    printf("System is NOT in safe state. Deadlock detected!\n");
    printf("Deadlocked processes: ");
    for (int i = 0; i < n; i++)
        if (finish[i] == 0)
            printf("P%d ", i);
    printf("\n");
}

return 0;
}

```

Output:

```
Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0
System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1
```

Lab Program 6

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

Program:

```
#include <stdio.h>
```

```
void firstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i]) {
                alloc[i] = j;
                used[j] = 1;
                break;
            }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
```

```
void bestFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
```

```

    used[i] = 0;
for (int i = 0; i < n; i++)
    alloc[i] = -1;

for (int i = 0; i < n; i++) {
    int best = -1;
    for (int j = 0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            if (best == -1 || blocks[j] < blocks[best])
                best = j;
    if (best != -1) {
        alloc[i] = best;
        used[best] = 1;
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i])
                if (worst == -1 || blocks[j] > blocks[worst])
                    worst = j;
        if (worst != -1) {

```

```

        alloc[i] = worst;
        used[worst] = 1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main() {

    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int processes[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processes[i]);

    firstFit(blocks, m, processes, n);
    bestFit(blocks, m, processes, n);
    worstFit(blocks, m, processes, n);

    return 0;
}

```

}

Output:

```
Name: STAWAN HITESH RUIWALE
USN:1WA24CS241
```

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426
```

```
--- First Fit ---
```

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

```
--- Best Fit ---
```

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

```
--- Worst Fit ---
```

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	4
4	426	Not Allocated

```
stavvv@Stavs-MacBook-Air OS_LAB_PGMS %
```


Lab Program 7

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

Program:

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
#define MAX_FRAMES 10
```

```
#define MAX_PAGES 50
```

```
// Helper: check if page is already in frames
```

```
bool isPresent(int frames[], int n, int page) {  
    for (int i = 0; i < n; i++)  
        if (frames[i] == page)  
            return true;  
    return false;  
}
```

```
void printSummary(const char *algo, int n, int misses) {
```

```
    int hits = n - misses;  
    printf("\n[%s Summary]\n", algo);  
    printf("Total Accesses : %d\n", n);  
    printf("Total Hits    : %d\n", hits);  
    printf("Total Misses   : %d\n", misses);  
    printf("Hit Ratio: %.2f%%\n", (hits * 100.0) / n);  
    printf("Miss Ratio: %.2f%%\n", (miss * 100.0) / n);  
}
```

```
// FIFO Page Replacement
```

```
void fifo(int pages[], int n, int numFrames) {  
    int frames[MAX_FRAMES];  
    int misses = 0, pointer = 0;
```

```
    for (int i = 0; i < numFrames; i++)  
        frames[i] = -1;
```

```

printf("\n--- FIFO ---\n");
printf("%-5s | Frames\n", "Page");

for (int i = 0; i < n; i++) {
    printf("%-5d | ", pages[i]);

    if (!isPresent(frames, numFrames, pages[i])) {
        frames[pointer] = pages[i];
        pointer = (pointer + 1) % numFrames;
        misses++;
        for (int j = 0; j < numFrames; j++)
            (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
        printf(" *MISS*");
    } else {
        for (int j = 0; j < numFrames; j++)
            (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
        printf(" *HIT*");
    }
    printf("\n");
}
printSummary("FIFO", n, misses);
}

```

```

// LRU Page Replacement
void lru(int pages[], int n, int numFrames) {
    int frames[MAX_FRAMES];
    int lastUsed[MAX_FRAMES];
    int misses = 0;

    for (int i = 0; i < numFrames; i++) {
        frames[i] = -1;
        lastUsed[i] = 0;
    }

    printf("\n--- LRU ---\n");
    printf("%-5s | Frames\n", "Page");

    for (int i = 0; i < n; i++) {
        printf("%-5d | ", pages[i]);
    }
}

```

```

if (!isPresent(frames, numFrames, pages[i])) {
    int lruIdx = 0;
    for (int j = 1; j < numFrames; j++)
        if (lastUsed[j] < lastUsed[lruIdx])
            lruIdx = j;

    frames[lruIdx] = pages[i];
    lastUsed[lruIdx] = i + 1;
    misses++;

    for (int j = 0; j < numFrames; j++)
        (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
    printf(" *MISS*");
} else {
    for (int j = 0; j < numFrames; j++)
        if (frames[j] == pages[i]) {
            lastUsed[j] = i + 1;
            break;
        }

    for (int j = 0; j < numFrames; j++)
        (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
    printf(" *HIT*");
}
printf("\n");
}
printSummary("LRU", n, misses);
}

```

// Optimal Page Replacement

```

int findOptimalVictim(int frames[], int numFrames, int pages[], int n,
                    int current) {
    int farthest = -1, victimIdx = 0;

    for (int i = 0; i < numFrames; i++) {
        int nextUse = n;
        for (int j = current + 1; j < n; j++) {
            if (pages[j] == frames[i]) {
                nextUse = j;
                break;
            }
        }
    }
}

```

```

    }
}
if (nextUse > farthest) {
    farthest = nextUse;
    victimIdx = i;
}
}
return victimIdx;
}

void optimal(int pages[], int n, int numFrames) {
    int frames[MAX_FRAMES];
    int misses = 0, filled = 0;

    for (int i = 0; i < numFrames; i++)
        frames[i] = -1;

    printf("\n--- Optimal ---\n");
    printf("%-5s | Frames\n", "Page");

    for (int i = 0; i < n; i++) {
        printf("%-5d | ", pages[i]);

        if (!isPresent(frames, numFrames, pages[i])) {
            if (filled < numFrames)
                frames[filled++] = pages[i];
            else {
                int victim = findOptimalVictim(frames, numFrames, pages, n, i);
                frames[victim] = pages[i];
            }
            misses++;

            for (int j = 0; j < numFrames; j++)
                (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
            printf(" *MISS*");
        } else {
            for (int j = 0; j < numFrames; j++)
                (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
            printf(" *HIT*");
        }
        printf("\n");
    }
}

```

```
    }  
    printSummary("Optimal", n, misses);  
}
```

```
int main() {
```

```
    printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");
```

```
    int pages[MAX_PAGES], n, numFrames;
```

```
    printf("Enter number of pages: ");  
    scanf("%d", &n);
```

```
    printf("Enter the reference string: ");  
    for (int i = 0; i < n; i++)  
        scanf("%d", &pages[i]);
```

```
    printf("Enter number of frames: ");  
    scanf("%d", &numFrames);
```

```
    fifo(pages, n, numFrames);  
    lru(pages, n, numFrames);  
    optimal(pages, n, numFrames);
```

```
    return 0;  
}
```

Output

Name: STAWAN HITESH RUIWALE
USN:1WA24CS241

Enter number of pages: 12

Enter the reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Enter number of frames: 3

--- FIFO ---

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 3 1	*MISS*
0	2 3 0	*MISS*
4	4 3 0	*MISS*
2	4 2 0	*MISS*
3	4 2 3	*MISS*
0	0 2 3	*MISS*
3	0 2 3	*HIT*

--- LRU ---

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 0 3	*MISS*
0	2 0 3	*HIT*
4	4 0 3	*MISS*
2	4 0 2	*MISS*
3	4 3 2	*MISS*
0	0 3 2	*MISS*
3	0 3 2	*HIT*

[LRU Summary]

Total Accesses : 12
Total Hits : 3
Total Misses : 9
Hit/Miss Ratio : 33.33%

--- Optimal ---

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 0 3	*MISS*
0	2 0 3	*HIT*
4	2 4 3	*MISS*
2	2 4 3	*HIT*
3	2 4 3	*HIT*
0	0 4 3	*MISS*
3	0 4 3	*HIT*

[Optimal Summary]

Total Accesses : 12
Total Hits : 5
Total Misses : 7
Hit/Miss Ratio : 71.43%